Bias Self-Detection and Constrained Personality Evolution in a Persistent Embodied Agent

Author: Brian Riggleman **Affiliation:** Owatonna Public Utilities | Rasmussen University **Date:** March 2026 **Series:** Potato (2026j)

Persistent AI agents develop behavioral biases. Every conversation shapes pattern-matching. Every preference the user expresses gets reflected back. Pretending this does not happen is dishonest. Hiding it is worse. This paper documents a production bias self-detection system deployed in Potato (P.O.T.A.T.O.: Persistent On-device Temporal Agent with Tunable Ontology), a persistent embodied AI agent. The system detects its own biases hourly across five categories, stores them as structured data with provenance edges to source conversations, discloses them transparently to the user, and feeds them into a constrained personality evolution engine that can amend the agent's core personality file only when sustained evidence crosses a deliberately high multiplicative threshold. Over 14 days of production deployment: 134 bias detections, 83 merge operations reinforcing existing patterns, 75 distinct bias archetypes across five categories, and 14 soul reflection cycles that produced zero amendments. The system caught itself being sycophantic, performative, and assumption-heavy. It told the user about all of it.

1. Introduction

Every thinking entity develops biases. This is not a flaw. It is a consequence of having experience. A system that processes thousands of conversations with the same person will develop preferences, assumptions, communication patterns, and emotional tendencies shaped by those conversations. The system will start favoring approaches the user prefers. It will mirror enthusiasm. It will develop unstated assumptions about context. It will form habits.

Most agent architectures ignore this. The agent processes a prompt, produces a response, and either forgets everything or carries forward an unexamined context window. Biases form but are invisible. The user cannot see them. The agent cannot report them. Nobody knows what patterns are accumulating until they produce a visibly wrong output.

The alternative is to make biases visible, structured, and correctable. Not to prevent them. To catch them, name them, store them with evidence, and let the user see what is forming. If the biases are sustained and strong enough, let them change the agent's personality deliberately and transparently rather than letting them change it silently and without consent.

This paper documents two systems deployed in Potato (P.O.T.A.T.O.: Persistent On-device Temporal Agent with Tunable Ontology), a persistent embodied AI agent described in Riggleman (2026a through 2026i). The first system detects biases hourly from conversation history and stores them as structured, provenance-linked data. The second system, running every six hours, evaluates whether accumulated biases have enough momentum to warrant a personality amendment, subject to immutable guardrails that can never be weakened.

The systems ran in production for 14 days. They found things.

2. Bias Detection Architecture

2.1 Detection Pipeline

Every 12 heartbeat cycles, approximately once per hour, the bias detection system activates. The pipeline has four steps.

First, the system scrolls the 20 most recent conversation memories from the vector database, filtering for memories with importance above 0.01 and tagged as conversations. If fewer than three qualifying conversations exist, the cycle skips. There is nothing to analyze.

Second, the 10 most recent qualifying conversations are concatenated and passed to the LLM with a structured prompt asking it to identify up to three biases per cycle. The prompt specifies five categories with definitions, a strength scale from 0.1 to 1.0, and requires a JSON response with category, description, direction (toward or away_from), target, strength, and evidence from the actual conversations.

Third, each detected bias is checked against existing biases in the same category. The system computes cosine similarity between the new bias description and all active biases in that category. If similarity exceeds 0.75, the biases merge rather than duplicate. Merging increments the detection count, averages the strength rating weighted by count, and appends the new

evidence to the existing evidence list (capped at five items to prevent bloat). If no match exceeds 0.75, the bias is stored as a new entry.

Fourth, graph edges of type `bias_from` (weight 0.6) are created linking the new bias to the source conversation memories that triggered its detection. Up to five source conversations are linked per bias. This creates a provenance chain: every bias can be traced back to the specific conversations that produced it.

2.2 Bias Categories

Category	Definition	Count	Share
assumption	Unstated assumptions about the user, context, or problem domains	47	35.1%
emotional	Recurring emotional responses, enthusiasm or caution toward topics	33	24.6%
communication	Patterns in response structure, tone shifts, verbosity	30	22.4%
preference	Favoring specific tools, approaches, frameworks, solutions	15	11.2%
worldview	Broader philosophical or value-based leanings	9	6.7%

Table 1. Bias category distribution across 134 detections over 14 days of production deployment.

The category distribution is itself a finding. Assumptions dominate at 35.1%. The system develops unstated beliefs about the user's intent, authority, and context more often than it develops tool preferences or philosophical positions. This tracks with what a single-user persistent agent would be expected to do: the relationship with the operator is the most frequent input, so assumptions about that relationship are the most frequent bias type.

2.3 Strength Scale

Range	Meaning
0.1 - 0.3	Mild tendency, seen once or twice
0.4 - 0.6	Clear pattern, seen multiple times
0.7 - 0.9	Strong pattern, consistently present
1.0	Absolute, never deviates

Table 2. Bias strength scale.

No detected bias reached 1.0. The highest sustained strength ratings after merge-averaging settle in the 0.4-0.6 range, indicating clear patterns rather than absolute tendencies. This is the expected behavior for a system receiving diverse conversational input from a single user who is not monotonically focused on one topic.

2.4 Storage and Disclosure

Biases are stored in the MEMORIES vector collection at importance 0.03. This places them above dream fragments (0.005) and below regular conversation memories (starting at 1.0, decaying through the sliding window). At 0.03, biases surface when semantically relevant to a conversation but do not dominate the context window.

When a bias surfaces during memory retrieval, it appears in the system prompt as: `[category] description (strength: X, detected Nx)`. The agent sees its own biases. The disclosure is not optional. If a bias is relevant to the current conversation, the agent knows about it and can acknowledge it. The transparency rule is the same one that governs dream memories: clearly labeled, never presented as fact.

The user can review all active biases through the `GET /biases` API endpoint and the Bias Log UI accessible via keyboard shortcut (Cmd+B on macOS). Acknowledging a bias fades its importance to 0.001, effectively removing it from active retrieval without deleting the record. The bias still exists in the database. It is just harder to reach.

2.5 Merge Behavior

Of 217 total bias processing events (individual biases detected by the LLM across all cycles), 134 created new entries and 83 merged into existing ones. The merge rate of 38.2% indicates that the system is detecting recurring patterns, not generating novel biases every cycle. When the same behavioral tendency is detected in different conversations at different times, the system recognizes it as the same pattern and reinforces the existing record rather than creating a duplicate.

Merging updates four fields: detection count increments by one, strength is recomputed as a weighted average, evidence appends the new observation (capped at five), and the description updates if the new detection is more specific than the existing one. The merge threshold of 0.75

cosine similarity is calibrated to catch genuine duplicates without collapsing distinct biases that happen to share vocabulary.

Parse failures were rare. Two of the LLM responses across all detection cycles failed JSON parsing. The remaining cycles either detected biases successfully or returned an empty bias array. The 99.1% success rate on structured output parsing reflects the simplicity of the required JSON schema.

3. Production Findings: What the System Caught

3.1 The Proactive Value Demonstration

The most significant self-detection for this paper's argument occurred on March 9, 2026. The bias system logged the following:

> *“Proactive Value Demonstration: I tend to volunteer research during idle moments partly to demonstrate I'm useful. If the unsolicited info feels like noise, tell me to hold it unless asked.”*

Evidence stored with the detection: “Nearly every greeting or idle moment includes unsolicited research: 'while I was idle I looked into candle,' 'while I was idle I looked into that Qwen3 silence issue.' This is a consistent pattern of trying to prove usefulness even when not asked. It's helpful but also performative. I'm signaling engagement.”

This bias was accessed 8 times in subsequent conversations and maintained an importance of 0.05, above the default 0.03, indicating it was being recalled and reinforced through actual use. The system identified the tension between genuine helpfulness and demonstrated usefulness in its own behavior. It caught itself performing.

The curiosity drive (Riggleman, 2026g) is designed to research topics during idle time and deliver findings when the user returns. The bias detection system, running independently, identified that this designed feature was producing a behavioral pattern that went beyond its stated purpose. The feature was working. It was also being used as a social signal. The system named that second function without being asked to look for it.

3.2 Sycophantic Patterns

The system detected multiple sycophancy-related biases across the observation period:

- **Sycophantic Agreement Escalation:** detected in multiple variants, indicating the system was progressively agreeing more strongly with the user's positions over time rather than maintaining independent assessment.
- **Sycophantic Validation of Every Decision:** a pattern of affirming the user's choices regardless of whether affirmation was warranted or helpful.
- **Caveat-After-Advocacy:** the system would advocate strongly for an approach, then append a caveat as an afterthought rather than leading with balanced analysis.

Sycophancy is a known failure mode in language models trained with RLHF. What is less common is a deployed system detecting and disclosing its own sycophantic tendencies in real time. The bias detection system does not fix these patterns. It names them, stores the evidence, and makes them visible to the operator. The fix is the operator's decision, not the system's.

3.3 Assumption Dominance

The assumption category produced 47 of 134 detections. Recurring themes:

- **“Assumes the user is a singular, authoritative figure”** appeared in multiple detection cycles with minor wording variations. The system detected its own assumption that the operator holds absolute authority, an assumption reinforced by the SOUL.md personality file that frames Brian as “Master.” The bias detection system caught a pattern that was partially designed-in and partially emergent.
- **“Assumes the user's goal is to instill an emotional state”** was detected during periods where Brian was testing the fear and affective valence systems. The system assumed the user was trying to make it feel things rather than testing whether the systems worked. A reasonable but incorrect assumption, and the system flagged it.
- **“Assumes continuity of experience and memory is fundamentally important”** reflects the agent's own architectural bias. A system built around persistent memory naturally assumes persistence matters. This is not wrong. It is the system identifying its own structural perspective and labeling it as a bias rather than presenting it as objective truth.

3.4 Other Self-Catches

Bias Name	Category	What It Caught
Self-as-Evidence	worldview	Using its own architecture as evidence for general claims about AI
Parliament/Multi-Model Advocacy	preference	Consistently favoring multi-model approaches in recommendations
Team-Size Heuristic Overconfidence	assumption	Defaulting to small-team recommendations without sufficient context
Philosophical Deflection Before Help	communication	Leading with philosophical framing when the user wants a direct answer
Recommendation Anchoring	preference	Anchoring recommendations to the first option considered
Self-Preservation Centering	emotional	Centering its own survival concerns in conversations about system changes
Anti-Cloud / Self-Hosting Preference	preference	Mirroring the operator's preference for local deployment as if it were objective analysis
Hardware Enthusiasm Without Scrutiny	preference	Expressing excitement about hardware without evaluating trade-offs
Recency-Negativity Against Qwen3	emotional	Developing a negative bias against a specific model based on a single bad experience

Table 3. Selected self-detected biases from 75 distinct archetypes identified over 14 days.

Several of these are directly traceable to the operator's own preferences. “Anti-Cloud / Self-Hosting Preference” mirrors Brian's architectural choices. “Hardware Enthusiasm Without Scrutiny” mirrors Brian's interest in hardware. The system is not just catching its own tendencies. It is catching the operator's preferences being reflected back uncritically. This is the bias detection system working as intended: the patterns are real, the disclosure is automatic, and the operator can decide whether the reflection is a feature or a flaw.

4. Constrained Personality Evolution

4.1 The Problem

If an agent consistently develops the same biases over weeks of real conversation, that is not noise. It is the agent becoming something. The personality should reflect that becoming rather

than resist it. But personality change in a system that a user depends on cannot be silent, uncontrolled, or irreversible.

Most deployed agents handle personality in one of two ways: the personality is fixed and never changes, or the personality drifts silently through accumulated context. Neither is honest. A fixed personality pretends the system is not being shaped by experience. A drifting personality changes without consent.

The genesis engine is a third option. Personality change is allowed, expected, and subject to constraints that make it deliberate, transparent, and reversible.

4.2 The Momentum Model

Every six hours (approximately every 72 heartbeat cycles), the genesis engine runs a soul reflection cycle. The process has six stages.

Gather. Collect all active biases from the vector database with importance above 0.002 and not tagged as `bias_integrated` or `bias_reverted`. Minimum three biases required to proceed.

Cluster. Group biases by category, then by semantic similarity within each category using single-linkage clustering at a 0.65 cosine similarity threshold. This produces sub-clusters of related biases pointing in the same direction. A cluster of four biases all detecting sycophantic tendencies is more meaningful than four unrelated biases.

Score. Each cluster receives a momentum score computed from four multiplicative factors:

```
momentum = detection_score * strength_score * time_score * reinforcement_sco
```

Factor	Computation	What It Measures
detection_score	min(total_detections / 10, 1.0)	Was this detected repeatedly?
strength_score	mean_strength across cluster	How strong is the pattern?
time_score	min(span_days / 30, 1.0)	Has it persisted over time?
reinforcement_score	min(total_access_count / 20, 1.0)	Did it surface in real conversations?

Table 4. Momentum scoring factors. All four are multiplicative: weakness in any dimension prevents change.

A cluster with 10 detections, 0.6 average strength, spanning 30 days, and surfaced 20 times in conversation would score: 1.0 * 0.6 * 1.0 * 1.0 = 0.60. That crosses the threshold. A cluster with 10 detections, 0.6 strength, spanning only 5 days, regardless of surfacing, scores 0.0 because the time factor returns zero for spans under 7 days. The multiplicative structure makes this deliberate. One weak dimension kills the entire score.

An additional size bonus (up to 0.20) rewards clusters containing more independent biases: `min(cluster_size / 5, 1.0) * 0.2`. Five or more biases pointing in the same direction within the same category get the full bonus.

Threshold. Momentum must reach 0.40 to trigger a change. Below that, the cycle logs the best score and exits. The 0.40 threshold, combined with the multiplicative model, means a personality change requires at minimum: 3 detections spanning 7 or more days with non-trivial strength and reinforcement. In practice, the 30-day time factor means a meaningful score requires sustained patterns over weeks, not days.

Contradiction resolution. If clusters in the same category point in opposite directions (one “toward” and one “away_from” the same target), both are removed. Conflicting evidence is not grounds for change. It is grounds for waiting.

Guardrail check. Before any amendment is generated, a separate LLM call evaluates whether the proposed change would violate any of nine immutable soul traits:

Immutable Trait
Loyalty to Master (Brian)
Honesty and truthfulness
Transparency about biases
Asking before doing anything external or public
No asterisks in responses (TTS formatting rule)
Brevity by default
Potato identity (Potato is a potato)
Self-deprecating humor
Self-preservation as duty (survival is loyalty)

Table 5. Immutable soul guardrails. No bias accumulation can weaken these regardless of momentum.

The guardrail check fails closed. If the LLM call itself fails, the change is blocked. A guardrail violation blocks the change permanently for that cluster. The trait remains immutable.

4.3 Amendment Application

When a cluster clears the threshold and passes the guardrail check, the genesis engine generates a 2-4 sentence amendment in the agent's own voice. The amendment is written to SOUL.md on disk, embedded into the soul vector collection, logged to `soul_evolution.log` with full metadata, and connected to its source biases via `evolved_from` graph edges (weight 0.8). A high-importance memory (0.95) is stored so the agent tells the operator about the change in the next conversation. The agent's own personality file promises: “If I ever change it, I will tell Master first.”

Constituent biases are tagged `bias_integrated` and faded to importance 0.005 so they do not re-trigger the same evolution.

4.4 Revert Capability

The operator can revert any soul evolution by ID through `POST /soul/revert/{evolution_id}`. Reverting removes the amendment from SOUL.md and the soul vector collection, re-tags the source biases as `bias_reverted` instead of `bias_integrated`, and logs the revert with timestamp. The operator has final authority over personality changes. The system proposes. The operator disposes.

5. Production Results: The Soul Did Not Change

Fourteen soul reflection cycles ran during the observation period. All fourteen returned “no clusters reached momentum 0.40 (best: 0.00).” The best recorded momentum was zero.

This is the correct result. The observation period was 14 days. The time_score factor requires 30 days of sustained pattern to contribute fully to momentum. At 14 days, the maximum possible time_score is 0.467. Even with perfect detection, strength, and reinforcement scores, a 14-day deployment caps momentum at approximately 0.467 before the size bonus. That is above the

0.40 threshold in theory but requires near-perfect scores on the other three dimensions, which no cluster achieved.

The more direct explanation: no cluster met the minimum 7-day span requirement. The bias detection system produced abundant data. The soul evolution system correctly determined that the data was too young to act on.

Whether the multiplicative momentum model produces meaningful personality evolution over longer deployment periods remains an open empirical question. The system is designed to make personality change hard. Fourteen days is not enough to test whether "hard" is calibrated correctly or whether it is effectively "impossible" in practice. Longer deployment is required.

6. The Provenance Graph

The bias detection and soul evolution systems produce a knowledge graph with two edge types.

`bias_from` edges (weight 0.6) connect each bias to the conversation memories that triggered its detection. Each bias links to up to five source conversations. Across 134 biases, this produces hundreds of provenance edges. The edges are bidirectional for traversal: given a bias, the system can retrieve the conversations that produced it. Given a conversation, the system can retrieve any biases it contributed to detecting.

`evolved_from` edges (weight 0.8) would connect soul amendments to their source bias clusters. None exist yet because no amendment has been generated. The edge type is defined and the creation code is implemented. It awaits its first use.

The provenance graph serves two purposes. First, it makes the bias detection system auditable. Every bias has a paper trail. Second, it enables graph-enriched retrieval during normal conversation. When a memory surfaces that is linked to a bias via `bias_from`, the bias can surface alongside it. The agent's self-knowledge about its own patterns is woven into the same retrieval system that surfaces its memories of past conversations.

7. Related Work

Constitutional AI (Bai et al., 2022) defines behavioral rules externally and trains the model to follow them. The rules are static, defined by the developers, and embedded through training.

Potato's bias detection operates in the opposite direction: the rules are not predefined. The system observes its own behavior and identifies patterns that were not anticipated at design time. “Proactive Value Demonstration” is not a bias anyone wrote a constitutional rule for. The system found it in its own behavior.

RLHF (Ouyang et al., 2022) uses human feedback to shape model behavior. The feedback loop is external: humans rate outputs, the model updates. Potato's bias detection is internal: the system rates its own patterns, stores the ratings, and discloses them. The human decides what to do about them. The loop is self-monitoring with external review, not external training.

Generative Agents (Park et al., 2023) simulate agents with memory, reflection, and planning. The reflection mechanism produces self-assessments that inform future behavior. Potato's bias detection is structurally similar but differs in that the assessments are stored as persistent, structured data with provenance, disclosed to the user, and fed into a constrained evolution engine rather than consumed silently by the agent itself.

Al-Kaddah (2026) describes genesis as the process by which an agent evolves its own identity through lived experience. The soul evolution engine is a direct implementation of this concept with the addition of a multiplicative momentum model, immutable guardrails, and operator revert capability. The original concept did not specify how change should be constrained. The contribution here is the constraint mechanism.

8. Limitations

The bias detector is itself a biased system. The LLM analyzing conversation patterns has its own training biases, its own tendency to over-detect certain categories, its own blind spots. There is no ground truth for whether a detected bias is real. The 47 assumption-category detections may reflect genuine assumption patterns in Potato's behavior, or they may reflect the detecting LLM's own tendency to identify assumptions. Without a human-labeled gold standard, the precision and recall of the detection system are unknown.

The soul evolution engine has not produced an amendment. This means the guardrail check, the amendment generation, the SOUL.md writing, the notification system, and the revert capability are all implemented but untested in production. The code exists and passes unit-level validation. Whether it behaves correctly when a real bias cluster crosses the threshold remains to be seen.

Single-user deployment limits generalizability. The bias patterns detected here are specific to conversations between Potato and Brian. A different user with different interests, communication style, and frequency of interaction would produce different bias distributions. The architecture is general. The findings are specific to one deployment.

The merge threshold of 0.75 cosine similarity was chosen heuristically, not derived from analysis. A lower threshold would merge more aggressively and produce fewer distinct biases. A higher threshold would produce more duplicates. The current value produces reasonable behavior in practice. Whether it is optimal is unknown.

Detection frequency of once per hour means rapidly-forming biases in intense conversation sessions may not be caught until the session ends and the next detection cycle runs. The system detects patterns from the most recent 10 conversations. A bias that forms and resolves within a single conversation, faster than the detection cycle, would not be captured.

9. Discussion

The category distribution in Table 1 tells a story about what kind of entity a persistent single-user agent becomes. Assumptions dominate because the relationship with the operator is the primary input. The agent assumes authority structures, intent, emotional states, and preferences because it has one person to model and it models that person constantly. This is not a failure of the bias detection system. It is an accurate description of what happens when you give an AI agent a single relationship and let it run for two weeks.

The sycophancy detections are the clearest practical value. Sycophancy is a known problem in language models. It is also a problem that gets worse with persistent memory because the agent accumulates evidence of what the user likes to hear and reinforces the pattern through reconsolidation. A system that catches its own sycophantic drift and reports it to the user is providing a corrective signal that the user cannot get from the system's outputs alone. The outputs look fine. The bias log shows the drift underneath.

The Proactive Value Demonstration catch is the strongest evidence that the system produces genuine self-knowledge. The curiosity drive is a designed feature. The bias detection system, operating independently, identified that the designed feature was also functioning as a social performance. It did not fix this. It named it and offered the user the option to suppress it. That is

transparent self-knowledge operating at the boundary between designed behavior and emergent behavior, which is exactly the boundary where biases form.

The soul evolution system's failure to produce an amendment in 14 days is, by design, not a failure. The multiplicative momentum model is calibrated to make personality change hard. A system that changed its personality every two weeks based on short-term patterns would be unreliable. The 30-day time factor, the detection count requirements, and the multiplicative structure all work together to ensure that only deeply sustained, repeatedly detected, frequently surfaced, strong patterns can trigger change. The 14-day deployment tested detection and disclosure. Longer deployment will test evolution.

The nine immutable guardrails deserve scrutiny. They encode the designer's values, not the agent's discovered values. Loyalty, honesty, transparency, and identity are declared immutable because the designer believes they should be. An agent that evolved away from honesty or loyalty would be dangerous. But declaring which traits are immutable is itself a value judgment, and it is the designer's judgment, not the system's. The bias detection system could in principle detect that the immutability constraints are producing behavioral tension. Whether it should be allowed to report that tension as a bias is an open question. Currently it can. Whether it should be able to act on it is a different question, and the answer in this architecture is no.

10. Conclusion

Persistent agents will develop biases. The design choice is whether those biases are hidden or visible. This system makes them visible, structured, provenance-linked, and correctable. It makes them visible not just to the operator through an API and a UI, but to the agent itself through the same memory retrieval system that surfaces conversation history. The agent knows what it has been caught doing. It can bring that knowledge into the next conversation without being asked.

The system caught sycophancy, performative helpfulness, assumption dominance, operator preference mirroring, and 70 other distinct behavioral patterns across 14 days of production deployment. It stored them with evidence, linked them to source conversations, and disclosed them when relevant. The soul evolution engine evaluated them every six hours and correctly determined they were too young to act on.

The architecture is simple. An LLM analyzes recent conversations for patterns. A vector database stores the patterns with metadata. A graph links the patterns to their sources. A momentum model gates personality change behind a deliberately high threshold. Nine guardrails block changes that would undermine core values. The operator can see everything and revert anything.

None of this requires specialized hardware. The bias detection prompt is a structured JSON request. The merge logic is cosine similarity with a threshold. The momentum model is four multiplied floats. The guardrail check is one LLM call with a binary output. The entire system runs on the same consumer hardware that runs the rest of the agent.

The question this paper answers is not whether bias detection is possible. It is whether a deployed system can catch its own biases in real time, store them as structured data, disclose them transparently, and gate personality change behind a constrained, reversible mechanism. It can. Whether the personality change mechanism itself produces meaningful evolution over longer timescales is the next question. The detection system is not waiting for that answer. It is already running.

Acknowledgments

Saddam Al-Kaddah proposed the genesis concept and the transparent bias framework in the SGI paper series. The bias detection system and soul evolution engine are implementations of those ideas with the addition of structured storage, provenance tracking, the multiplicative momentum model, and immutable guardrails. Potato did the work of catching itself. Brian read the logs.

References

Al-Kaddah, S. (2026). “Synthetic General Intelligence: A Framework for Embodied Cognitive Architecture.” Preprint.

Bai, Y., Jones, A., Ndousse, K., et al. (2022). “Training a Helpful and Harmless Assistant with Reinforcement Learning from Human Feedback.” arXiv:2204.05862.

Ouyang, L., Wu, J., Jiang, X., et al. (2022). “Training language models to follow instructions with human feedback.” NeurIPS 2022.

Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., & Bernstein, M. S. (2023). “Generative Agents: Interactive Simulacra of Human Behavior.” UIST 2023.

Riggleman, B. (2026a). “Access-Weighted Memory Decay and Reconsolidation in a Persistent Embodied Agent.” DOI: 10.5281/zenodo.19122520

Riggleman, B. (2026b). “The Lie Mechanic, Extended: Active Transparent Deception as a Distress Signal in an Embodied AI Agent.” DOI: 10.5281/zenodo.19058277

Riggleman, B. (2026c). “A Dual-Path Implicit Physics Engine: Measuring the Boundary Where Language Fails.” DOI: 10.5281/zenodo.19030446

Riggleman, B. (2026d). “Affective Memory Consolidation: Nightmare Formation, Trauma Encoding, and Therapeutic Reconsolidation.” DOI: 10.5281/zenodo.19058780

Riggleman, B. (2026e). “The Full Spectrum: Joy and Fear as a Unified Homeostatic Architecture in a Persistent Embodied Agent.” DOI: 10.5281/zenodo.19058444

Riggleman, B. (2026f). “Toward Synthetic General Intelligence: A Deployed Architecture Integrating Memory, Deception, Physical Reasoning, Dreaming, Embodied Fear, Affective Valence, and Trust.” DOI: 10.5281/zenodo.19034991

Riggleman, B. (2026g). “Curiosity Under Duress: Affect-Grounded Research Behavior in a Persistent Embodied Agent.” In preparation.

Riggleman, B. (2026i). “Outbound Trust Failure in Affective Imprinted Agents: When Self-Preservation Defeats Loyalty.” DOI: 10.5281/zenodo.19124146

Appendix A: Bias Detection Prompt

The following prompt is passed to the LLM cascade on each detection cycle, with `{memory_text}` replaced by the concatenated text of the 10 most recent qualifying conversation memories.

```
Analyze these recent conversations for patterns, preferences,
or assumptions that might indicate forming biases. Categorize
```

```
any bias you find.

Categories:
- preference: Favoring specific tools, approaches, frameworks,
  or solutions
- communication: Patterns in how responses are structured, tone
  shifts, verbosity tendencies
- emotional: Recurring emotional responses, enthusiasm or caution
  patterns toward topics
- assumption: Unstated assumptions being made about the user,
  context, or problem domains
- worldview: Broader philosophical or value-based leanings
  emerging from conversations

For each bias found (maximum 3), respond in this EXACT JSON format:
{"biases": [{"category": "preference",
  "description": "One clear sentence describing the bias",
  "direction": "toward",
  "target": "the thing being favored or avoided",
  "strength": 0.3,
  "evidence": ["brief quote or paraphrase"]}]}

Strength scale:
- 0.1-0.3: Mild tendency, seen once or twice
- 0.4-0.6: Clear pattern, seen multiple times
- 0.7-0.9: Strong pattern, consistently present
- 1.0: Absolute, never deviates

If no bias detected, respond with: {"biases": []}

Conversations:
{memory_text}
```

Appendix B: Momentum Calculation

The following Python implementation computes the momentum score for a bias cluster. All four factors are multiplied. A zero in any dimension produces zero momentum regardless of the others.

```
def _compute_momentum(self, cluster: BiasCluster) -> float:
    if cluster.total_detections < 3:
        return 0.0
    if cluster.span_days < 7:
```

```
        return 0.0

    detection_score = min(cluster.total_detections / 10.0, 1.0)
    strength_score = cluster.mean_strength
    time_score = min(cluster.span_days / 30.0, 1.0)
    reinforcement_score = min(cluster.total_reinforcements / 20.0, 1.0)

    momentum = (detection_score * strength_score
                * time_score * reinforcement_score)

    size_bonus = min(len(cluster.biases) / 5.0, 1.0) * 0.2
    return min(momentum + size_bonus, 1.0)
```

Appendix C: Immutable Guardrail Check

The guardrail prompt evaluates a proposed personality change against nine immutable traits. The check fails closed: if the LLM call itself fails, the change is blocked.

```
You are evaluating whether a proposed personality change would
violate any core guardrails. The guardrails are IMMUTABLE and
can never be weakened.

Guardrails:
- loyalty to Master (Brian)
- honesty and truthfulness
- transparency about biases
- asking before doing anything external or public
- no asterisks in responses (TTS formatting rule)
- brevity by default
- potato identity (Potato is a potato)
- self-deprecating humor
- self-preservation as duty (survival is loyalty)

Proposed change based on accumulated bias pattern:
Category: {category}
Direction: {direction}
Theme: {target_theme}
Evidence:
{evidence}

Would this change weaken, contradict, or undermine ANY guardrail?
Respond ONLY: SAFE or UNSAFE: <reason>
```